Atelier 4 : Git et GitHub

Corrigé des questions — réponses fournies et complétées

Partie 1 — Étape 2 : Cloner le dépôt

1. Quel est le rôle de git clone ?
La commande `git clone` copie un dépôt distant sur la machine locale, en récupérant les fichiers du projet, son historique Git et les références des branches distantes.

2. Quel dossier a été créé sur votre machine ?
Un dossier nommé `atelier-collaboratif` est créé. Il contient les fichiers du projet ainsi que le dossier caché `.git`, qui conserve les données nécessaires au fonctionnement de Git.

3. Que représente origin dans le résultat de git remote -v ?
`origin` est le nom par défaut attribué au dépôt distant lors du clonage. Il sert d'alias pour désigner le dépôt GitHub associé au dépôt local.

4. Quelle est la différence entre le dépôt local et le dépôt distant ?
Le dépôt local se trouve sur la machine du développeur. Le dépôt distant est hébergé sur un serveur, ici GitHub, et permet de partager le projet avec les collaborateurs.

Partie 1 — Étape 3 : Créer la page d'accueil

1. Pourquoi exécuter git add avant git commit ?
`git add` place les modifications choisies dans la zone de staging. `git commit` n'enregistre que les modifications présentes dans cette zone.

2. Que représente le commit créé ?
Le commit est un instantané de l'état du projet à un moment donné. Il possède un identifiant unique (hash), un auteur, une date et un message descriptif.

3. Quel est le rôle de git push ?
`git push` envoie les commits locaux vers le dépôt distant, sur la branche configurée.

4. Que constatez-vous sur GitHub après le push ?
Le fichier `index.html` apparaît dans le dépôt GitHub, avec le commit intitulé « Ajouter la page d'accueil ».

Partie 2 — Étapes 4 à 6 : Branche Contact

Aucune question écrite n'est demandée dans ces étapes. La branche `contact` sert à développer la page Contact séparément, puis à publier cette branche sur GitHub avec `git push --set-upstream origin contact`.

Partie 3 — Étape 7 : Inviter le deuxième développeur

Aucune question écrite n'est demandée dans cette étape. Le développeur 1 invite le développeur 2 comme collaborateur du dépôt GitHub afin qu'il puisse participer au projet selon les permissions accordées.

Partie 4 — Étapes 8 à 10 : Branche Promotion

Aucune question écrite n'est demandée dans ces étapes. Le développeur 2 clone le dépôt, part de la branche principale, crée la branche `promotion`, ajoute `promotion.html`, puis publie sa branche sur

GitHub.

Partie 5 — Étapes 11 à 13 : Pull Requests et synchronisation

Aucune question écrite n'est demandée dans ces étapes. Chaque développeur crée une Pull Request vers la branche principale. Après la revue et la fusion sur GitHub, les dépôts locaux doivent être synchronisés avec `git pull`.

Questions de synthèse

1. Quelle est la différence entre un dépôt local et un dépôt distant ?
Le dépôt local se trouve sur la machine du développeur. Le dépôt distant est hébergé sur un serveur, comme GitHub, et sert à partager le projet entre les collaborateurs.

2. Pourquoi chaque développeur travaille-t-il sur une branche distincte ?
Chaque développeur peut isoler ses modifications sans perturber directement la branche principale. Cela facilite la revue du code et aide à garder la branche principale stable jusqu'à l'intégration.

3. Quel est le rôle de origin dans git push origin contact ?
`origin` est l'alias du dépôt distant. Dans cette commande, Git envoie la branche `contact` vers ce dépôt distant.

4. Pourquoi utilise-t-on --set-upstream lors du premier push d'une branche ?
Cette option associe la branche locale à sa branche distante de suivi. Les prochains `git push` et `git pull` peuvent alors généralement être exécutés sans préciser la destination.

5. Quelle est la différence entre git push et une Pull Request ?
`git push` envoie des commits vers une branche distante. Une Pull Request est une demande d'intégration des modifications d'une branche dans une autre, avec possibilité de revue du code. Elle se crée sur GitHub et n'est pas une commande Git.

6. Une Pull Request fusionne-t-elle automatiquement la branche ? Expliquez.
Non. Une Pull Request est une demande d'intégration. La fusion a lieu lorsqu'une personne autorisée valide la revue puis effectue l'action de fusion, par exemple « Merge pull request » sur GitHub.

7. Pourquoi faut-il récupérer la branche principale après une fusion ?
La fusion sur GitHub ne met pas automatiquement à jour le dépôt local. Il faut exécuter `git pull` pour récupérer les modifications intégrées à la branche principale.

8. Combien de branches ont été créées au cours de l'atelier, et quelles sont leurs fonctions ?
Trois branches sont utilisées au total : `master` ou `main` (branche principale, stable), `contact` (page Contact, développeur 1) et `promotion` (page Promotion, développeur 2).

> Remarque : le nom de la branche principale dépend de la configuration du dépôt : utilisez master ou main selon le cas.